

ASSIGNMENT

Real-Life Applications and Demonstration of Bubble Sort and Binary Search

Department of Artificial Intelligence and Data Science
KPR Institute of Engineering and Technology

Submitted By:

Name: Santhosh S

Roll No: 25AD149

GitHub Repository: <https://github.com/Sant-spec/algorithm-real-life-applications>

1. Introduction

An algorithm is a step-by-step procedure or set of rules designed to solve a specific problem or perform a computation. In computer science, algorithms form the core logic of software applications, ensuring execution consistency, efficiency, and predictability.

Sorting and searching are fundamental operations in data processing. Sorting involves ordering data based on predefined criteria (e.g., numerical or alphabetical order), whereas searching involves locating target records within structured datasets. Efficient implementation of these operations directly impacts application performance in real-world systems.

This report covers two foundational algorithms: **Bubble Sort** (a comparison-based sorting method) and **Binary Search** (a divide-and-conquer searching method). It includes algorithm definitions, operational logic, Python implementations, execution steps, real-world application analysis, time/space complexity evaluations, and a structured VivaVoce preparation guide.

2. Objectives

- To understand the theoretical concepts and working mechanics of Bubble Sort and Binary Search.
- To implement both algorithms cleanly using the Python programming language.
- To evaluate real-life industrial and domain applications for each algorithm.
- To trace algorithm execution state and memory changes step-by-step using Python Tutor.
- To perform comparative analysis of time and space complexities.

3. Algorithm 1: Bubble Sort

3.1 Definition

Bubble Sort is a comparison-based sorting algorithm. It operates by iteratively scanning a list, comparing adjacent pairs, and swapping them if they are out of order. This process repeats until a complete pass occurs without any swaps, indicating the list is fully sorted. The name originates from larger values gradually "bubbling" to the upper boundaries of the array.

3.2 Operational Logic

For an array of size n , the algorithm performs up to $n-1$ passes. In each pass, adjacent elements at index j and $j+1$ are evaluated. If $\text{array}[j] > \text{array}[j+1]$, an in-place swap occurs. After pass 1, the largest element reaches index $n-1$. After pass k , the k largest elements reside in their correct final positions.

3.3 Step-by-Step Execution Trace

Initial List: [5, 3, 8, 4, 2]

Pass 1:

- Compare 5 and 3 → $5 > 3$ → Swap → [3, 5, 8, 4, 2]
- Compare 5 and 8 → $5 < 8$ → No swap → [3, 5, 8, 4, 2]
- Compare 8 and 4 → $8 > 4$ → Swap → [3, 5, 4, 8, 2]
- Compare 8 and 2 → $8 > 2$ → Swap → [3, 5, 4, 2, 8]

Pass 2:

- Compare 3 and 5 → No swap → [3, 5, 4, 2, 8]
- Compare 5 and 4 → Swap → [3, 4, 5, 2, 8]
- Compare 5 and 2 → Swap → [3, 4, 2, 5, 8]

Pass 3:

- Compare 3 and 4 → No swap → [3, 4, 2, 5, 8]
- Compare 4 and 2 → Swap → [3, 2, 4, 5, 8]

Pass 4:

- Compare 3 and 2 → Swap → [2, 3, 4, 5, 8]

Final Sorted Array: [2, 3, 4, 5, 8]

3.4 Pseudocode

```
BEGIN
  INPUT: list of n elements
  FOR i = 0 TO n-2:
    FOR j = 0 TO n-2-i:
      IF list[j] > list[j+1]:
        SWAP list[j] and list[j+1]
      END IF
    END FOR
  END FOR
  OUTPUT: sorted list
END
```

3.5 Python Implementation

```
# Bubble Sort Implementation - Student Marks Ordering

marks = [72, 45, 89, 33, 60]

print("=== BUBBLE SORT - Student Marks ===")
print("Original Marks:", marks)
print()

n = len(marks)

for i in range(n - 1):
    print(f"--- Pass {i + 1} ---")
    for j in range(n - 1 - i):
        if marks[j] > marks[j + 1]:
            marks[j], marks[j + 1] = marks[j + 1], marks[j]
            print(f"  Swapped: index {j} and {j+1} -> {marks}")
        else:
            print(f"  No swap : index {j} and {j+1} -> {marks}")
    print(f"  After Pass {i + 1}: {marks}")
    print()

print("=== Sorted Marks (Ascending Order) ===")
print("Final Sorted Marks:", marks)
```

3.6 Program Output

Output Link:<https://tinyurl.com/28k2h8lv>

```
=== BUBBLE SORT - Student Marks ===
Original Marks: [72, 45, 89, 33, 60]

--- Pass 1 ---
  Swapped: index 0 and 1 -> [45, 72, 89, 33, 60]
  No swap : index 1 and 2 -> [45, 72, 89, 33, 60]
```

```

    Swapped: index 2 and 3 -> [45, 72, 33, 89, 60]
    Swapped: index 3 and 4 -> [45, 72, 33, 60, 89]
    After Pass 1: [45, 72, 33, 60, 89]

--- Pass 2 ---
    No swap : index 0 and 1 -> [45, 72, 33, 60, 89]
    Swapped: index 1 and 2 -> [45, 33, 72, 60, 89]
    Swapped: index 2 and 3 -> [45, 33, 60, 72, 89]
    After Pass 2: [45, 33, 60, 72, 89]

--- Pass 3 ---
    Swapped: index 0 and 1 -> [33, 45, 60, 72, 89]
    No swap : index 1 and 2 -> [33, 45, 60, 72, 89]
    After Pass 3: [33, 45, 60, 72, 89]

--- Pass 4 ---
    No swap : index 0 and 1 -> [33, 45, 60, 72, 89]
    After Pass 4: [33, 45, 60, 72, 89]

=== Sorted Marks (Ascending Order) ===
Final Sorted Marks: [33, 45, 60, 72, 89]

```

3.7 Time and Space Complexity Analysis

Case	Time Complexity	Description
Best Case	$O(n)$	Occurs when the input list is already sorted (with optimized early exit check).
Average Case	$O(n^2)$	Occurs when elements are in random distribution.
Worst Case	$O(n^2)$	Occurs when the list is sorted in reverse order.
Space Complexity	$O(1)$	Operates strictly in-place without auxiliary allocation.

3.8 Real-World Application: Academic Rank Generation

Scenario: Academic institutions process internal assessment scores (e.g., [72, 45, 89, 33, 60]) to publish structured merit lists and establish percentile rankings.

Application: Sorting provides direct accessibility to minimum/maximum scores and boundary classifications. While Bubble Sort is inefficient for enterprise-scale databases, its in-place property and minimal overhead make it suitable for small-scale embedded hardware or introductory processing modules.

4. Python Tutor Execution Logs: Bubble Sort

- **Initial State:** Unsorted array loaded into frame memory; outer loop pointer $i=0$ initialized.
- **First Swap (Pass 1):** Comparison at indices 0 and 1 (72 vs 45) triggers conditional swap block.
- **Completion of Pass 1:** Primary maximum element (89) locked at terminal index 4.
- **Intermediate Passes:** Inner loop range dynamically reduces to $n - 1 - i$, skipping sorted right-side elements.
- **Final State:** Array state matches [33, 45, 60, 72, 89]; loop terminates.

5. Algorithm 2: Binary Search

5.1 Definition

Binary Search is a logarithmically scalable search algorithm that locates a target key within a previously sorted dataset. It employs a divide-and-conquer paradigm, repeatedly bisecting the search interval until the key is found or the range resolves to zero.

5.2 Prerequisites

The target list **must be strictly sorted** prior to executing Binary Search. Invoking Binary Search on unsorted data breaks the directional elimination logic and produces incorrect results.

5.3 Operational Logic

Pointers low and high frame the active search interval. In each iteration, $mid = (low + high) // 2$ is evaluated:

- If $array[mid] == target \rightarrow$ Key located at index mid.
- If $array[mid] < target \rightarrow$ Adjust $low = mid + 1$ (eliminate left partition).
- If $array[mid] > target \rightarrow$ Adjust $high = mid - 1$ (eliminate right partition).

5.4 Execution Trace Example

Sorted Input: [10, 20, 30, 40, 50, 60, 70] | **Target Key:** 50

Step	Low	High	Mid Index	Mid Value	Decision / Action
1	0	6	3	40	40 < 50 → Shift search window to right half (low = 4)
2	4	6	5	60	60 > 50 → Shift search window to left half (high = 4)
3	4	4	4	50	50 == 50 → Target identified at Index 4

5.5 Pseudocode

```

BEGIN
  INPUT: sorted list, target value
  SET low = 0, high = length of list - 1
  WHILE low <= high:
    SET mid = (low + high) // 2
    IF list[mid] == target:
      PRINT "Found at index mid"
      STOP
    ELSE IF list[mid] < target:
      SET low = mid + 1
    ELSE:
      SET high = mid - 1
  END WHILE
  PRINT "Element not found"
END

```

5.6 Python Implementation

```
# Binary Search Implementation - Student Registration Lookup

roll_numbers = [101, 105, 110, 115, 120, 125, 130]
target = 120

print("=== BINARY SEARCH - Student Record Search ===")
print("Sorted Roll Numbers:", roll_numbers)
print(f"Searching for Roll Number: {target}\n")

low = 0
high = len(roll_numbers) - 1
found = False
step = 1

while low <= high:
    mid = (low + high) // 2
    print(f"Step {step}:")
    print(f"  Low = {low}, High = {high}, Mid = {mid}")
    print(f"  Middle element = roll_numbers[{mid}] = {roll_numbers[mid]}")

    if roll_numbers[mid] == target:
        print(f"  {roll_numbers[mid]} == {target} --> FOUND!\n")
        print(f"  Roll Number {target} found at index {mid} (Position {mid + 1}).")
        found = True
        break
    elif roll_numbers[mid] < target:
        print(f"  {roll_numbers[mid]} < {target} --> Search RIGHT half\n")
        low = mid + 1
    else:
        print(f"  {roll_numbers[mid]} > {target} --> Search LEFT half\n")
        high = mid - 1
    step += 1

if not found:
    print(f"Roll Number {target} was NOT found in the list.")
```

5.7 Program Output

Output Link: <https://tinyurl.com/252fhs62>

```
=== BINARY SEARCH - Student Record Search ===
Sorted Roll Numbers: [101, 105, 110, 115, 120, 125, 130]
Searching for Roll Number: 120

Step 1:
  Low = 0, High = 6, Mid = 3
  Middle element = roll_numbers[3] = 115
  115 < 120 --> Search RIGHT half
```

Step 2:

```
Low = 4, High = 6, Mid = 5
Middle element = roll_numbers[5] = 125
125 > 120 --> Search LEFT half
```

Step 3:

```
Low = 4, High = 4, Mid = 4
Middle element = roll_numbers[4] = 120
120 == 120 --> FOUND!
```

Roll Number 120 found at index 4 (Position 5).

5.8 Time and Space Complexity Analysis

Case	Time Complexity	Description
Best Case	$O(1)$	Target element matches initial mid calculated element.
Average Case	$O(\log n)$	Subdivision cuts problem space exponentially in half each iteration.
Worst Case	$O(\log n)$	Occurs when target is at boundary limits or non-existent in array.
Space Complexity	$O(1)$	Iterative approach consumes constant space.

5.9 Real-World Application: Educational Database Indexing

Scenario: System searches student record databases containing over 100,000 entries ordered by primary roll numbers.

Efficiency Comparison: Linear search requires up to 100,000 sequential comparisons in worst-case lookups. Binary Search isolates any record within a maximum of 17 iterations ($\log_2 100,000 \approx 16.6$), dramatically reducing server I/O overhead.

6. Python Tutor Execution Logs: Binary Search

- **Initial State:** Pointers set at low=0 and high=6; target established as 120.
- **Iteration 1:** Mid index computed as 3 (value 115); conditional update sets low=4.
- **Iteration 2:** Mid index recomputed as 5 (value 125); conditional update sets high=4.
- **Iteration 3:** Range converges to low=4, high=4, mid=4; matching key found; return status true.

7. Comparative Analysis

Feature / Property	Bubble Sort	Binary Search
Primary Purpose	Ordering unsorted collections	Searching keys in sorted collections
Input Requirement	Accepts arbitrary/unsorted arrays	Requires pre-sorted arrays strictly
Core Operation	Pairwise adjacent swap comparisons	Interval bisection via mid-point checks
Best-Case Complexity	$O(n)$	$O(1)$
Worst-Case Complexity	$O(n^2)$	$O(\log n)$
Space Complexity	$O(1)$ In-place	$O(1)$ Iterative
Scalability Target	Small datasets / Pedagogical use	Enterprise database index lookups

8. Technical Repository Architecture

The standard submission archive contains the following structure:

```
algorithm-real-life-applications/  
├─ Bubble_Sort/  
│   └─ bubble_sort.py  
├─ Binary_Search/  
│   └─ binary_search.py  
├─ Screenshots/  
│   ├── bubble_sort_pyttutor.png  
│   └─ binary_search_pyttutor.png  
├─ Assignment_Report/  
│   └─ [ROLLNO]_[NAME].pdf  
  
└─ README.md
```

9. Conclusion

Bubble Sort and Binary Search highlight core algorithmic principles in computer science. Bubble Sort illustrates foundational sorting mechanics, nested iteration, and in-place transformations. Binary Search demonstrates how structure (sorted order) dramatically improves computational efficiency from $O(n)$ to $O(\log n)$. Understanding these foundational patterns builds the necessary baseline for designing scalable software systems.